Handwritten Digit Recognition

Computer Vision - Assignment 1 | Reproducible mini-project Logistic regression and a convolutional neural network

1. Objective and dataset

The objective is to classify an isolated handwritten digit into one of ten classes (0-9), and compare a linear baseline with a CNN under the same data budget. Mathematical symbols and multi-character expressions are outside this experiment. All reported results were obtained by executing the accompanying notebook.

Dataset provenance: this project uses public MNIST images, not a self-collected dataset. Therefore, the technical experiment is complete, but the assignment requirement to collect original handwriting remains unmet. A separate notebook template is included for a future writer-separated collection.

MNIST provides 28 x 28 grayscale images of handwritten digits [1]. We selected 200 unique images per class (2,000 total), using NumPy random seed 42. For each class, 170 images came from the official training partition and 30 from its test partition. The 170 training-partition images were divided into 140 training and 30 validation examples.

Partition	Per class	Total	Purpose
Training	140	1,400	Fit model parameters
Validation	30	300	Select C and CNN checkpoint
Test	30	300	Final held-out evaluation

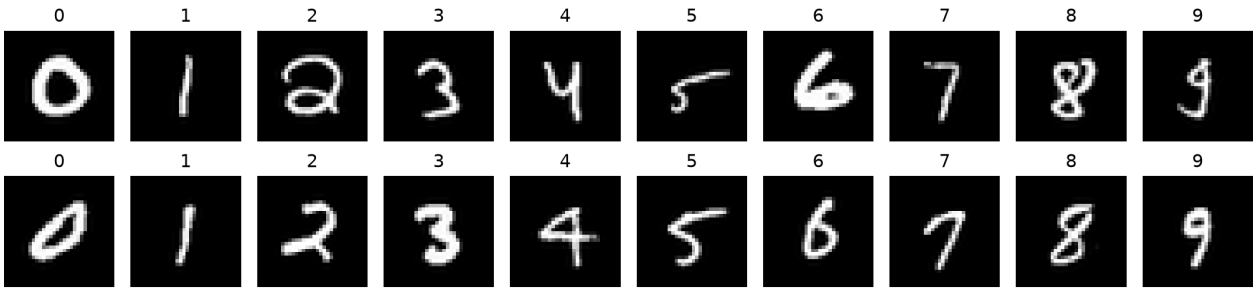


Figure 1. Two training samples per digit after preprocessing. These are public handwritten images, not synthetic fonts.

Each exported PNG is stored in a class folder. A manifest preserves the original partition and sample index, selected split, and pixel SHA-256. Exact pixel duplicates are excluded and audited; near-duplicates are not ruled out. The source archive does not include writer IDs, so writer independence of validation cannot be verified.

2. Preprocessing and models

Exported PNG files use dark ink on a light background. The shared preprocessing converts to grayscale, inverts intensities, fits the image into a 24 x 24 area without changing aspect ratio, and centers it on a black 32 x 32 canvas. Pixel values are divided by 255 to obtain [0,1] inputs. No statistics are estimated from validation or test images.

Training augmentation adds exactly one transformed copy per original image: random rotation between -10 and +10 degrees and translation up to 8% in each direction, with black fill. The resulting 2,800 training inputs are identical for both models. Validation and test samples are not augmented. Flips are excluded because they can alter the meaning of a digit.

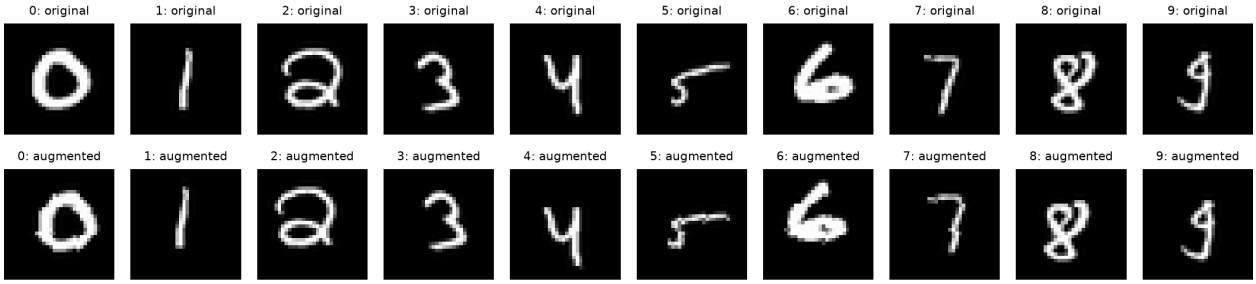


Figure 2. Original inputs (top) and their fixed augmented training copies (bottom).

Baseline model

Logistic regression uses 1,024 flattened pixel features, L2 regularization, and a maximum of 2,000 optimization iterations. C was selected from {0.1, 1, 10} using validation macro F1; the selected value was 0.1. Both models are evaluated on the same untouched test images.

Convolutional neural network

Block	Operation	Output
1	Conv 3x3, 32 channels; ReLU; max pool 2x2	32 x 16 x 16
2	Conv 3x3, 64 channels; ReLU; max pool 2x2	64 x 8 x 8
3	Conv 3x3, 128 channels; ReLU; max pool 2x2	128 x 4 x 4
Classifier	Flatten; dense 128; ReLU; dropout 0.4; dense 10	10 logits

The CNN has 356,234 trainable parameters. It uses cross-entropy loss and Adam (learning rate 0.001, weight decay 0.0001), batch size 64, and at most 30 epochs. Early stopping allows six epochs without validation macro-F1 improvement. The run ended after 26 epochs; checkpoint 20 was selected. No test measurements informed checkpoint selection.

3. Held-out evaluation

Accuracy is the fraction of correctly classified images. Precision, recall, and F1 are calculated per class and then averaged equally across the ten classes (macro averaging). All classes have 30 test images. Zero-denominator scores are set to zero. Detailed per-class reports and individual predictions are exported as CSV files.

Model	Accuracy	Macro precision	Macro recall	Macro F1
Logistic regression	89.00%	89.11%	89.00%	88.89%
CNN	98.33%	98.47%	98.33%	98.35%

Logistic regression made 33 errors out of 300 test images; the CNN made 5. The CNN accuracy difference was +9.33 percentage points. This is a measured comparison for the selected subset and training run, not a guarantee that the same gap will hold on new handwriting.

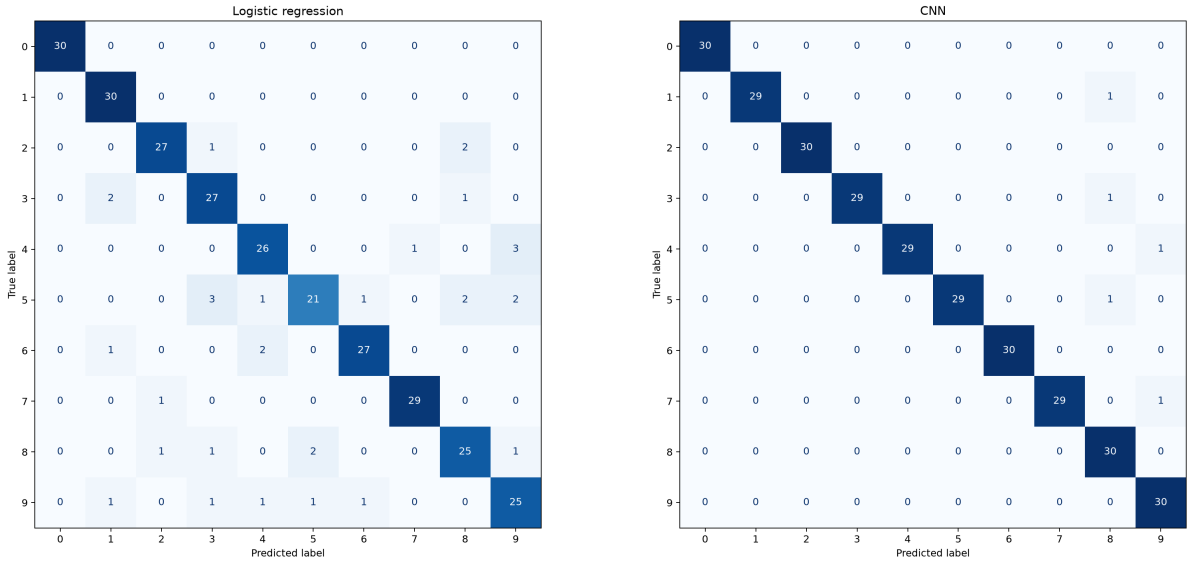


Figure 3. Test confusion matrices. Rows are true labels; columns are predicted labels. Diagonal cells are correct classifications. Counts in each row sum to 30.

Sampling uncertainty

Logistic regression: the image-level percentile bootstrap 95% accuracy interval is 85.3% to 92.3%.

CNN: the image-level percentile bootstrap 95% accuracy interval is 96.7% to 99.7%.

Using 5,000 paired bootstrap resamples, the accuracy-difference interval (CNN minus baseline) is +5.7 to +13.0 percentage points. The pairing preserves the fact that both models classify the same images. These intervals do not capture training-seed variability or writer clustering; they should be treated as descriptive uncertainty for this test subset.

4. Learning behavior and errors

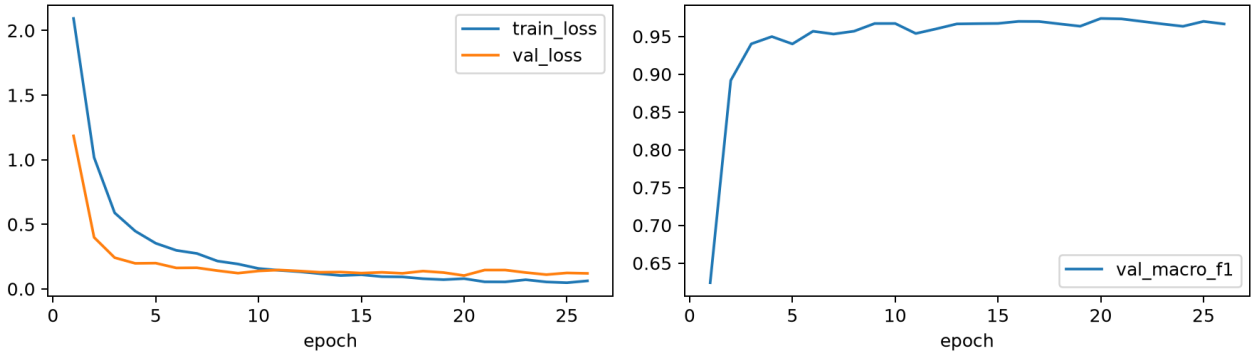


Figure 4. Training/validation cross-entropy and validation macro F1. The best validation-F1 checkpoint is restored before testing.

Validation macro F1 reached 97.37% at epoch 20. At that epoch, training loss was 0.0795 and validation loss was 0.1033. Training uses dropout and augmented images, while validation uses evaluation mode and original images, so the two loss curves are not directly equivalent estimates.

The most frequent CNN confusion pairs were: 1 predicted as 8 (1 images); 3 predicted as 8 (1 images); 4 predicted as 9 (1 images); 5 predicted as 8 (1 images). These counts come directly from held-out predictions. The error montage provides examples for qualitative inspection; appearance-based explanations remain hypotheses rather than established causes.

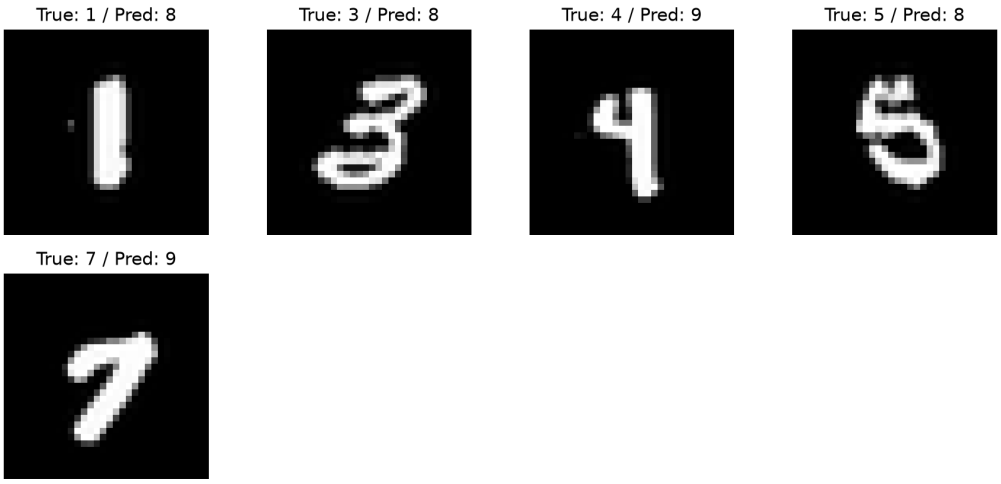


Figure 5. Up to twelve CNN test mistakes, in manifest order. Labels show the ground truth and prediction; empty panels mean fewer than twelve errors.

Errors on isolated digits can arise from ambiguous strokes, atypical shapes, or preprocessing effects. A future analysis should examine these factors on a larger, separately collected test set. This run does not include an augmentation ablation, so any improvement cannot be attributed specifically to augmentation.

5. Conclusion and reproducibility

On the balanced 300-image test subset, logistic regression achieved 89.00% accuracy and the CNN achieved 98.33%. The experiment demonstrates a complete pipeline from dataset auditing to model comparison and saved inference models. Its main limitation for this assignment is that the images were taken from MNIST rather than collected by the student.

Limitations and next steps

The conclusions are limited to ten digit classes, one random seed, 1,400 original training images, and a small test set. The validation split has no writer identifiers. MNIST inputs are clean and normalized, so performance on photographs, varied backgrounds, pen colors, or mathematical symbols is unmeasured. Model softmax scores are not calibrated confidence estimates.

To meet the self-collection requirement, collect 200 original images per class, for example from 20 writers producing ten examples of each digit. Keep writers w01-w14 in training, w15-w17 in validation, and w18-w20 in testing. Use the included self-collected template to rerun both models, and replace these benchmark results with the new measurements. Adding mathematical symbols requires new labeled classes and retraining; the current model predicts only digits.

Reproduction and delivered artifacts

The experiment used seed 42 and cpu execution. Baseline model selection took 3.2 seconds; CNN training and its learning-curve output took 56.3 seconds on this machine. Timing is hardware-dependent. Library versions are saved in results/environment.json and requirements.txt.

Install dependencies with python -m pip install -r requirements.txt. The included dataset is ready to use; python prepare_data.py rebuilds it from the source archive if necessary. Run python run_notebook.py to regenerate all experiment outputs. Run python predict.py PATH_TO_IMAGE for inference with the saved CNN, or python demo.py for a local drawing interface. Run python make_report.py after training to regenerate this report.

The package includes the executed notebook, 2,000 class-organized PNGs, provenance and split manifests, both saved models, evaluation CSVs, confusion matrices, learning curves, error examples, editable report text, and reproduction scripts.

References and data attribution

[1] Keras documentation. MNIST digits classification dataset. https://keras.io/api/datasets/mnist/ . MNIST copyright: Yann LeCun and Corinna Cortes; derived from NIST datasets. The selected, inverted PNG subset is distributed under CC BY-SA 3.0: https://creativecommons.org/licenses/by-sa/3.0/ .

[2] TensorFlow documentation. Load NumPy data. https://www.tensorflow.org/tutorials/load_data/numpy . Documents the MNIST NPZ download used by the reproduction script.

[3] Computer Vision Course Assignment - Dataset & Project. Instructor-provided Assignment1.pdf. Requirements: self-collected data, preprocessing and augmentation, baseline/improved models, evaluation, and a 3-5 page report.